

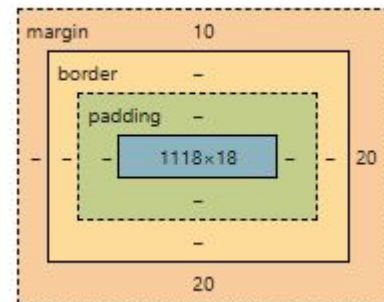
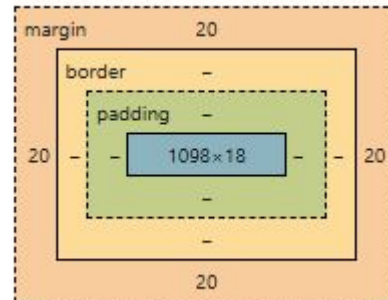
Cascading Style Sheets

CSS Margin

- Margins create space outside an element's border.
- Margins can be set for top, bottom, left, and right.
- Properties: `margin`, `margin-top`, `margin-right`, `margin-bottom`, `margin-left`.
- Values: `auto`, `px`, `%`, `em`, etc.

Example

```
div {  
  margin: 20px; /* All sides */  
  margin-top: 10px;  
  margin-left: auto;  
}
```



CSS Margin

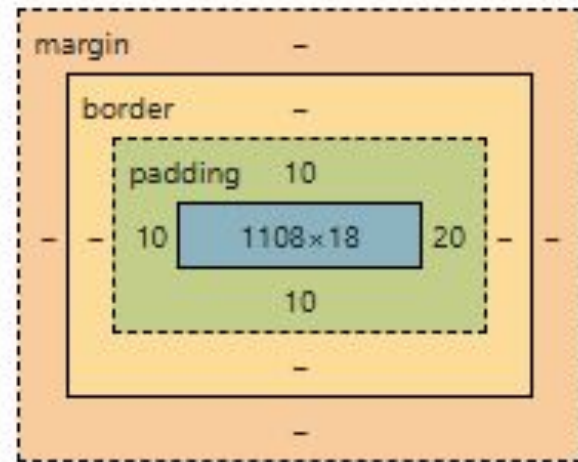
Unit	Type	Relative To	Use Case
auto	Automatic	Content/Parent size	Centering, automatic sizing.
px	Absolute	Fixed screen pixels	Fixed spacing, precise design.
%	Relative	Parent container	Fluid layouts, responsive designs.
em	Relative	Parent's font size	Scalable fonts, padding/margin.
vw	Relative	Viewport width	Responsive elements scaling to screen.

CSS Padding

- Padding creates space between an element's content and its border.
- Properties: `padding`, `padding-top`, `padding-right`, etc.
- Values: `px`, `%`, `em`.

Example

```
div {  
  padding: 10px; /* All sides */  
  padding-right: 20px;  
}
```

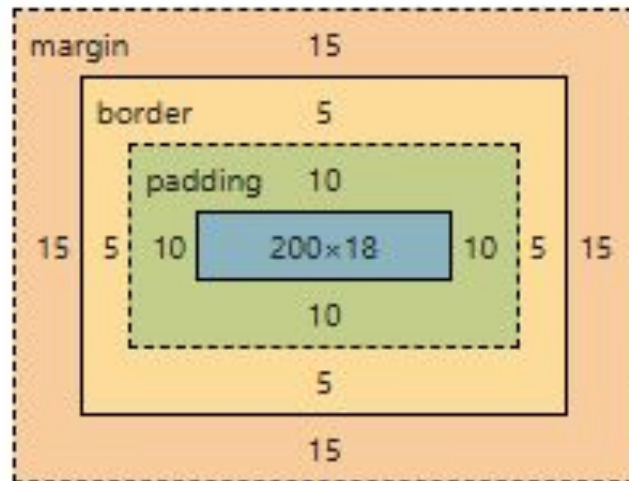


CSS Box Model

- Every element is a rectangular box.
- Box model includes:
 - Content
 - Padding
 - Border
 - Margin
- Total size = Content + Padding + Border + Margin

Example

```
div {  
  width: 200px;  
  padding: 10px;  
  border: 5px solid black;  
  margin: 15px;  
}
```



CSS Align

- Alignment options:
 - `text-align` for text (left, center, right).
 - `vertical-align` for inline elements.

Example

```
div {  
  width: 200px;  
  border: 1px solid;  
  text-align: center;  
}
```

Hey!

CSS Fonts

- Font properties control text appearance.
- Common properties:
 - `font-family`
 - `font-size`
 - `font-style`
 - `font-weight`
- Use web-safe fonts or Google Fonts.

Example

```
p {  
    font-family: Arial, sans-serif;  
    font-size: 16px;  
    font-weight: bold;  
}
```

CSS Links

- Use pseudo-classes to style links:
 - `:link`, `:visited`, `:hover`, `:active`.

Example

```
a:link { color: blue; }  
a:hover { color: red; }
```

CSS Lists

- Style lists using `list-style-type` and `list-style-position`.
- Remove default bullets using `list-style: none`.

Example

```
ul {  
  list-style-type: square;  
  list-style-position: outside;  
  width: 100px;  
}  
li {  
  border: 1px solid #000;  
}
```

- Window
- Linux

Example

```
ul {  
  list-style-type: disc;  
  list-style-position: inside;  
  width: 100px;  
}  
li {  
  border: 1px solid #000;  
}
```

- Window
- Linux

CSS Tables

- Style tables using:
 - `border`, `border-collapse`, `padding`, `text-align`, etc.

Example

```
table {  
  width: 100%;  
  border-collapse: collapse;  
  
}  
th, td {  
  border: 1px solid black;  
  padding: 8px;  
}
```

Windows	Linux	MacOS
---------	-------	-------

CSS Opacity

- The `opacity` property controls the transparency of an element.
- Value range: `0` (fully transparent) to `1` (fully opaque).

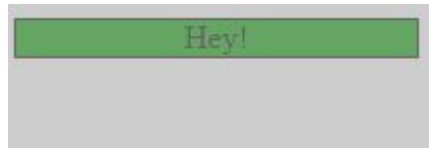
Example

```
div {  
  width: 200px;  
  border: 1px solid;  
  text-align: center;  
  opacity:1;  
  background-color: green;  
}
```



Example

```
div {  
  width: 200px;  
  border: 1px solid;  
  text-align: center;  
  opacity:0.5;  
  background-color: green;  
}
```



CSS Navigation Bar

- Use `ul`, `li`, and `a` tags to create nav bars.
- Style using `display: inline-block` or Flexbox.

Example

```
ul {  
  list-style-type: none;  
  margin: 0;  
  padding: 0;  
  overflow: hidden;  
}  
li {  
  float: left;  
}  
li a {  
  display: block;  
  padding: 8px;  
  background-color: #dddddd;  
  text-decoration: none;  
}
```



Home News Contact About

CSS Animation

- Use `@keyframes` to create animations.
- Apply animation using `animation` property.

Example

```
@keyframes example {  
  from { background-color: red; }  
  to { background-color: yellow; }  
}  
div {  
  animation: example 3s infinite;  
}
```

CSS Flexbox

- Flexbox (Flexible Box Layout) is a one-dimensional layout system in CSS.
- Used to align and distribute space among items within a container.
- It works on a single axis: main axis (horizontal) or cross axis (vertical).
- Makes layouts responsive and eliminates the need for floats and positioning hacks.

Key Terms:

- **Flex Container** (parent element).
- **Flex Items** (child elements).

CSS Flexbox

Container Properties:

- `display: flex` – Defines a flex container to enable Flexbox.
- `flex-direction` – Sets the direction of the main axis (row, column).
- `justify-content` – Aligns items along the main axis (start, center, space-between).
- `align-items` – Aligns items along the cross axis (start, center, stretch).
- `flex-wrap` – Allows items to wrap onto multiple lines.
- `align-content` – Aligns multiple lines of items on the cross axis.

Item Properties:

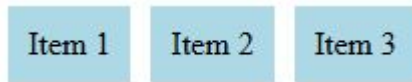
- `flex` – Shorthand for `flex-grow`, `flex-shrink`, and `flex-basis` to control item size.
- `align-self` – Aligns a single item independently on the cross axis.
- `order` – Controls the order of items within the container.

CSS Flexbox display: flex

- To use Flexbox, apply `display: flex` to a container.
- This turns child elements into **flex items**.

Example

```
<div class="container">
  <div class="item">Item 1</div>
  <div class="item">Item 2</div>
  <div class="item">Item 3</div>
</div>
<style>
  .container { display: flex; }
  .item {
    background: lightblue;
    margin: 5px; padding: 10px;
  }
</style>
```



CSS Flexbox flex-direction

Defines the direction of the main axis.

Values:

- `row` (default, horizontal left-to-right).
- `row-reverse`.
- `column` (vertical top-to-bottom).
- `column-reverse`.

Example

```
<div class="container">
  <div class="item">Item 1</div>
  <div class="item">Item 2</div>
  <div class="item">Item 3</div>
</div>
<style>
  .container { display: flex; flex-direction: row; }
  .item { background: lightblue; margin: 5px; padding: 10px; }
</style>
```

row

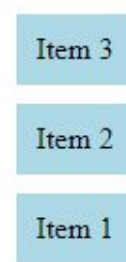
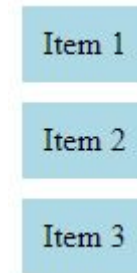


row-reverse



column-reverse

column



CSS Flexbox flex-direction

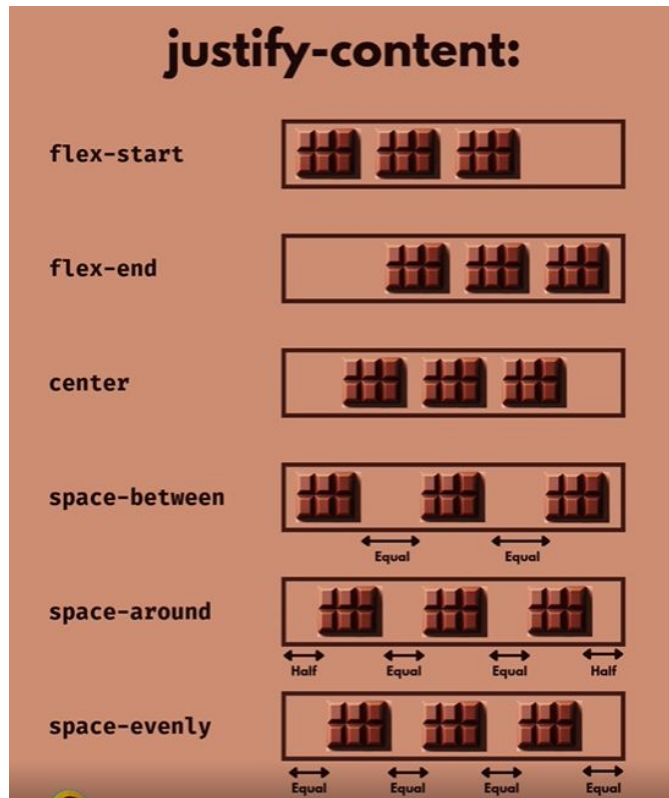
Aligns items along the **main axis**.

Values:

- `flex-start` (default).
- `flex-end`.
- `center`.
- `space-between`.
- `space-around`.
- `space-evenly`.

Example

```
<style>  
    .container {  
        display: flex;  
        justify-content: space-between;  
    }  
</style>
```



CSS Flexbox align-items

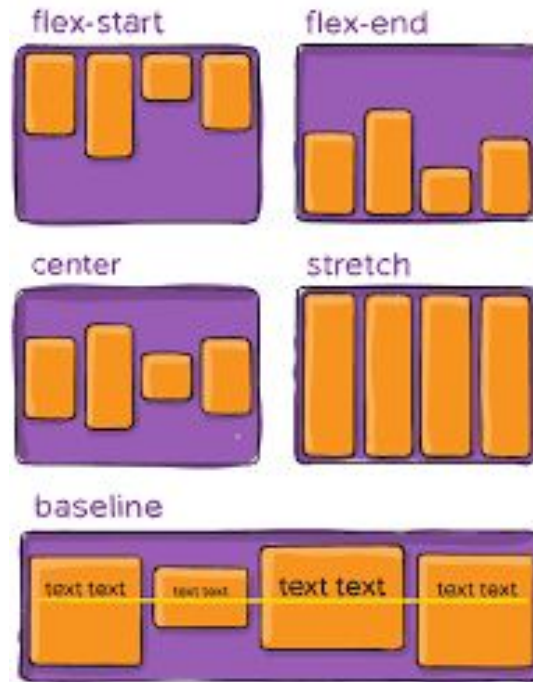
Aligns items along the **cross axis**.

Values:

- `stretch` (default).
- `flex-start`.
- `flex-end`.
- `center`.
- `baseline`.

Example

```
<style>  
  .container {  
    display: flex;  
    align-items: center;  
  }  
</style>
```



Thank you